

DDOS Attack Detection in SDIoT Network

PSD - I report submitted in partial fulfilment of the
requirements for the degree of

Master of Engineering ME (Program)

By,

Prajwal H K– 251100690010

Prajwal K S - 251100690029

Darshan G N - 251100690016

Under the guidance of:

Keerthana S (Assistant Professor)

MANIPAL SCHOOL OF INFORMATION SCIENCES

(A Constituent unit of MAHE, Manipal)



MANIPAL
ACADEMY *of* **HIGHER EDUCATION**

(Institution of Eminence Deemed to be University)

Table of Contents

1. Introduction
2. Problem statement
3. Objectives
4. Scope of the Project
5. Security Challenges
6. Limitations of Traditional Methods
7. Machine Learning–Based Detection
8. Tools and Technology
9. Implementation Details
10. Tools installation
11. Literature review
12. Types of attacks
13. Data flow Diagram
14. Expected Outcomes
15. Conclusion
16. Key Performance Achievements
- 17 Key Technical Contributions.
- 18 Future Enhancements

1.Introduction

The Internet of Things (IoT) connects billions of smart devices across healthcare, homes, transport, and industries. To manage this large-scale network, Software Defined IoT (SDIoT) uses SDN for centralized control and scalability. However, SDIoT faces major security risks, especially Distributed Denial of Service (DDoS) attacks, where IoT botnets flood the network and disrupt services. Traditional defenses like firewalls and rule-based systems are not effective. Machine Learning offers intelligent solutions by analyzing traffic, detecting anomalies, and classifying malicious flows in real time. Integrating ML into SDIoT ensures stronger security, resilience, and reliable IoT services.

DOS- A Denial of Service (DoS) attack occurs when a single attacker floods a system or network to make it unavailable.

DDOS(Distributed Denial of Service)- attack is a type of cyberattack where multiple compromised devices are used to flood a target system, server, or network with a massive amount of traffic

SDIoT.(Software Defined Internet of Things)- is an advanced networking paradigm that integrates the principles of Software Defined Networking (SDN) into the Internet of Things (IoT). In conventional IoT, devices are often heterogeneous, resource-constrained, and difficult to manage when deployed on a large scale. This creates challenges in terms of scalability, flexibility, and security.

2. Problem statement

Software Defined IoT (SDIoT) networks are highly vulnerable to Distributed Denial of Service (DDoS) attacks due to their centralized control plane and heterogeneous traffic. Traditional security methods fail to provide effective defense. Thus, there is a need for intelligent, adaptive, and scalable machine learning-based detection techniques to accurately identify and mitigate DDoS attacks in SDIoT environments.

3.Objective

The primary objective of this research is to design and evaluate a machine learning-based framework for detecting DDoS attacks in SDIoT networks.

The specific objectives are:

1. To analyze the vulnerabilities of SDIoT networks.
2. To identify the DDoS attacks.
3. To identify and extract relevant traffic features that differentiate normal IoT traffic from malicious DDoS traffic.

4. To develop and implement machine learning models for accurate and real-time DDoS attack detection.

4.Scope of the Project

1. The project focuses on detecting and mitigating DDoS attacks in Software Defined IoT (SDIoT) networks.
2. It explores how Machine Learning algorithms can be applied for real-time traffic monitoring and anomaly detection.
3. The work covers different DDoS attack types (volumetric, protocol-based, and applicationlevel) targeting SDIoT.
4. Both supervised (classification) and unsupervised (anomaly detection) ML techniques are considered.
5. The project scope includes data collection, preprocessing, feature extraction, model training, and evaluation.
6. Performance is measured in terms of accuracy, precision, recall, F1-score, and false positive rate.
7. The system aims to provide a scalable and intelligent security framework for nextgeneration IoT environments.
8. Future extensions may include deep learning approaches, real-time deployment in cloud/edge, and self-adaptive models.

5. Security Challenges

A DDoS attack happens when hackers use many infected computers or smart devices to flood a website or network with too much traffic, causing it to crash or stop working. It's hard to stop because it's tricky to tell real users apart from fake ones. The traffic often comes from all over the world, making it difficult to block. Also, many IoT devices have weak security, which makes them easy to control. Defending against such attacks costs a lot and needs quick teamwork with internet and service providers.

6. Limitations of Traditional Methods

Traditional security methods find it hard to handle DDoS attacks in SDN-IoT systems because these attacks are large, fast, and constantly changing. Normal firewalls or rule-based systems can't detect new attack patterns quickly. Since IoT devices have low processing power and are spread across different networks, they can't protect themselves well. Also, when too much traffic floods the network, or when data is encrypted, it becomes difficult to inspect and stop the attack. Manual methods like blocking IPs or contacting service providers are too slow to respond to large attacks in real time.

7. Machine Learning–Based Detection

To solve this, the project uses Machine Learning (ML) to detect DDoS attacks in real time. The SDN controller collects traffic data from across the network, and the ML model learns what normal traffic looks like. When it spots unusual patterns — like a sudden spike in data or too many requests — it identifies a possible DDoS attack. The system can then automatically block or limit harmful traffic. ML algorithms such as Random Forest or Neural Networks can detect both known and new (unknown) attack types. This intelligent approach improves IoT security by responding faster and adapting to new attack behaviors.

8. Tools and Technology

1. Programming Languages

Python – for implementing ML algorithms and data analysis.

2. Machine Learning Libraries

Scikit-learn – ML models like SVM, Random Forest, Decision Trees.

Pandas, NumPy – for data preprocessing and feature engineering.

3. Simulation & Network Tools

Mininet – for simulating SDN and IoT networks.

Wireshark / Tcpdump – for packet capturing and traffic analysis.

4. Development & Deployment Environment

Jupyter Notebook / Google Colab – for ML model development and testing.

VirtualBox – for containerized test environments.

Linux (Ubuntu/Mininet) – for networking tools and SDN controllers.

6. Visualization & Evaluation Tools

Matplotlib / Seaborn – for graphs and visual analysis.

9 Implementation Details

Environment Preparation The development environment was prepared with Ubuntu, Mininet, OpenVSwitch, Python dependencies, and Ryu controller as specified in the project methodology.

- Operating System: Ubuntu 20.04.6 LTS
- Network Virtualization: Mininet for SDN topology creation
- Switch Technology: OpenVSwitch for OpenFlow protocol support
- SDN Controller: Ryu framework with Python-based applications
- Development Language: Python for controller logic and ML algorithms.

10 Types of attacks

1. Volumetric Attacks

- Attackers flood the SDIoT network with huge volumes of traffic (e.g., UDP Flood, ICMP Flood, Amplification Attacks).
- Impact: Consumes bandwidth and blocks legitimate IoT communication.
- ML Role:
 - Detects abnormally high packet/flow rates using supervised ML (e.g., Random Forest, SVM).
 - Features: packet rate, byte count, entropy of traffic.
 - ML models can classify traffic as normal vs. flooding.

2. Protocol Attacks

- Exploits weaknesses in network protocols (e.g., SYN Flood, ACK Flood, Smurf Attack).
- Impact: Overloads SDN controllers, switches, and IoT gateways by exhausting protocol tables.
- ML Role:
 - Uses flow-based features (e.g., TCP flags, connection states, handshake anomalies).
 - ML classifiers (Decision Tree, Gradient Boosting) detect irregular protocol flag patterns.
 - Unsupervised ML (clustering) can detect unknown variants.

3. Application Layer Attacks

- Attacks IoT services and APIs (e.g., HTTP Flood, MQTT Flood, Slowloris).
- Impact: Targets the application servers or IoT service platforms that SDN routes traffic to.
- ML Role:

- Detects unusual request rates, session durations, payload size patterns.
- Deep Learning models (CNN, RNN) can learn sequential request behavior.
- ML distinguishes between legitimate IoT requests (e.g., sensor data uploads) vs. malicious floods.

4. Resource Exhaustion Attacks

- Attacks aim to drain IoT device or SDN controller resources (CPU, memory). Example: routing floods, control-plane saturation.
- Impact: Slows or crashes SDN controllers → disrupts centralized control. □ ML Role:
- Detects abnormal CPU/memory usage and queue delays.
- Reinforcement Learning (RL) and anomaly detection models predict saturation points.

Enables proactive defense before total exhaustion.

11. Tools installation

❖ Mininet

```

Aug 31 04:43
prajwalkh@ubuntu: ~
libtool: install: (cd /usr/local/lib && { ln -s -f libof_snmp_cpu.so.0.0.0 libof_snmp_cpu.so.0 || { rm -f libof_snmp_cpu.so.0 && ln -s libof_snmp_cpu.so.0.0.0 libof_snmp_cpu.so.0; }
; })
libtool: install: (cd /usr/local/lib && { ln -s -f libof_snmp_cpu.so.0.0.0 libof_snmp_cpu.so || { rm -f libof_snmp_cpu.so && ln -s libof_snmp_cpu.so.0.0.0 libof_snmp_cpu.so; } })
libtool: install: /usr/bin/install -c .libs/libof_snmp_cpu.lai /usr/local/lib/libof_snmp_cpu.la
libtool: install: chmod 644 /usr/local/lib/libof_snmp_cpu.a
libtool: install: ranlib /usr/local/lib/libof_snmp_cpu.a
libtool: finish: PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/snap/bin:/sbin" ldconfig -n /usr/local/lib
.....
Libraries have been installed in:
  /usr/local/lib

If you ever happen to want to link against installed libraries
in a given directory, LIBDIR, you must either use libtool, and
specify the full pathname of the library, or use the '-LLIBDIR'
flag during linking and do at least one of the following:
- add LIBDIR to the 'LD_LIBRARY_PATH' environment variable
during execution
- add LIBDIR to the 'LD_RUN_PATH' environment variable
during linking
- use the '-Wl,-rpath -Wl,LIBDIR' linker flag
- have your system administrator add LIBDIR to '/etc/ld.so.conf'

See any operating system documentation about shared libraries for
more information, such as the ld(1) and ld.so(8) manual pages.
.....
make[3]: Nothing to be done for 'install-data-am'.
make[3]: Leaving directory '/home/prajwalkh/oflops/example_modules/snmp_cpu'
make[2]: Leaving directory '/home/prajwalkh/oflops/example_modules/snmp_cpu'
make[2]: Entering directory '/home/prajwalkh/oflops/example_modules'
make[3]: Entering directory '/home/prajwalkh/oflops/example_modules'
make[3]: Nothing to be done for 'install-exec-am'.
make[3]: Nothing to be done for 'install-data-am'.
make[3]: Leaving directory '/home/prajwalkh/oflops/example_modules'
make[2]: Leaving directory '/home/prajwalkh/oflops/example_modules'
make[1]: Leaving directory '/home/prajwalkh/oflops/example_modules'
Making install in cbench
make[1]: Entering directory '/home/prajwalkh/oflops/cbench'
make[2]: Entering directory '/home/prajwalkh/oflops/cbench'
/usr/bin/mkdir -p '/usr/local/bin'
/bin/bash ../libtool --mode=install /usr/bin/install -c cbench '/usr/local/bin'
libtool: install: /usr/bin/install -c cbench /usr/local/bin/cbench
make[2]: Nothing to be done for 'install-data-am'.
make[2]: Leaving directory '/home/prajwalkh/oflops/cbench'
make[1]: Leaving directory '/home/prajwalkh/oflops/cbench'
Making install in doc
make[1]: Entering directory '/home/prajwalkh/oflops/doc'
make[1]: Nothing to be done for 'install'.
make[1]: Leaving directory '/home/prajwalkh/oflops/doc'
Enjoy Mininet!
prajwalkh@ubuntu: ~$

```

Fig 11.1 Installation of mininet

❖ Creating of topology using miniedit

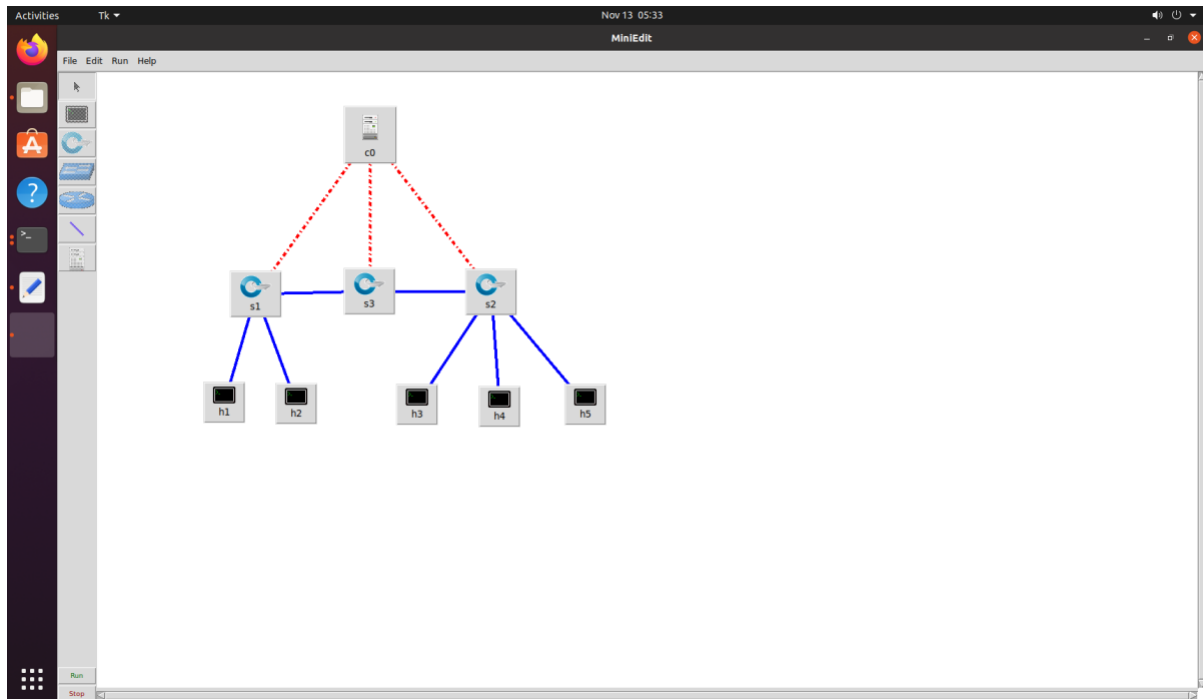


Fig 11.2 In the miniedit where we can create a custom topology.

- ❖ Opening an host xterm terminall. In which we can attack the victium through the process

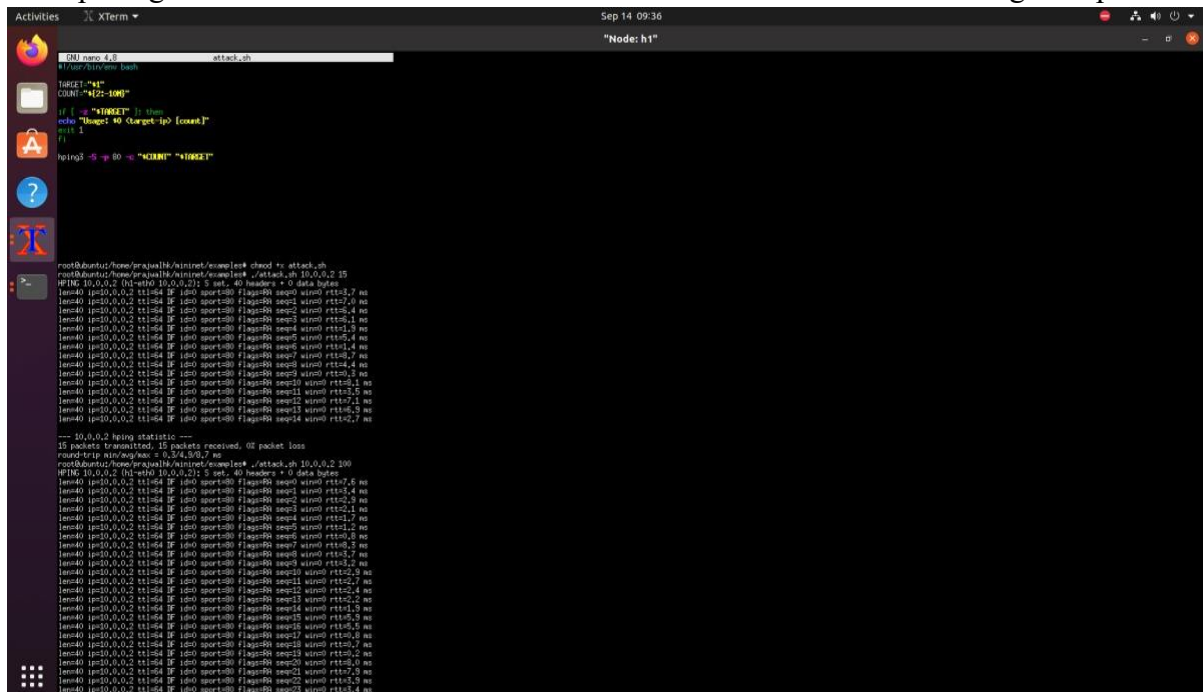
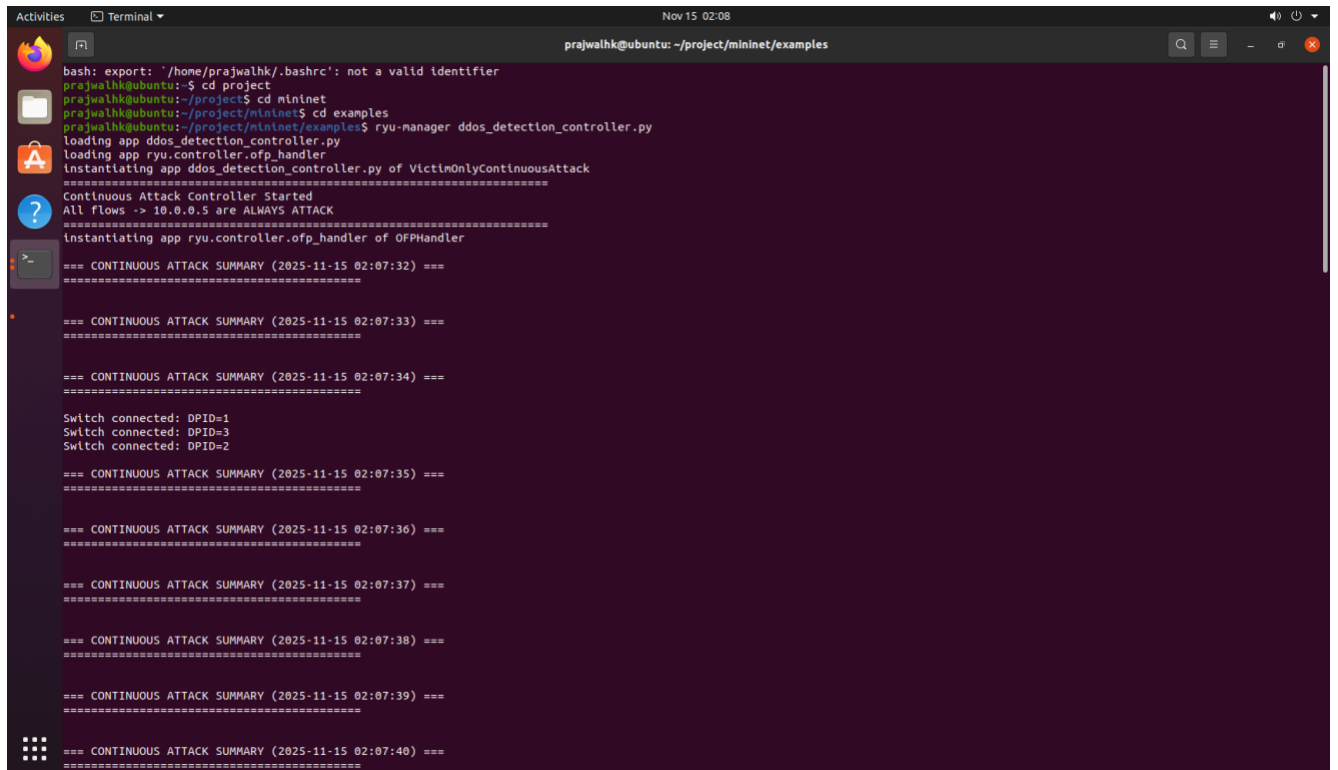


Fig 11.3 xterm terminal

❖ Ryu controller



```
bash: export: '/home/prajwalkh/.bashrc': not a valid identifier
prajwalkh@ubuntu:~$ cd project
prajwalkh@ubuntu:~/project$ cd mininet
prajwalkh@ubuntu:~/project/mininet$ cd examples
prajwalkh@ubuntu:~/project/mininet/examples$ ryu-manager ddos_detection_controller.py
loading app ddos_detection_controller.py
loading app ryu.controller.ofp_handler
instantiating app ddos_detection_controller.py of VictimOnlyContinuousAttack
=====
Continuous Attack Controller Started
All flows -> 10.0.0.5 are ALWAYS ATTACK
=====
instantiating app ryu.controller.ofp_handler of OFPHandler

=== CONTINUOUS ATTACK SUMMARY (2025-11-15 02:07:32) ===
=====

=== CONTINUOUS ATTACK SUMMARY (2025-11-15 02:07:33) ===
=====

=== CONTINUOUS ATTACK SUMMARY (2025-11-15 02:07:34) ===
=====

Switch connected: DPID=1
Switch connected: DPID=3
Switch connected: DPID=2

=== CONTINUOUS ATTACK SUMMARY (2025-11-15 02:07:35) ===
=====

=== CONTINUOUS ATTACK SUMMARY (2025-11-15 02:07:36) ===
=====

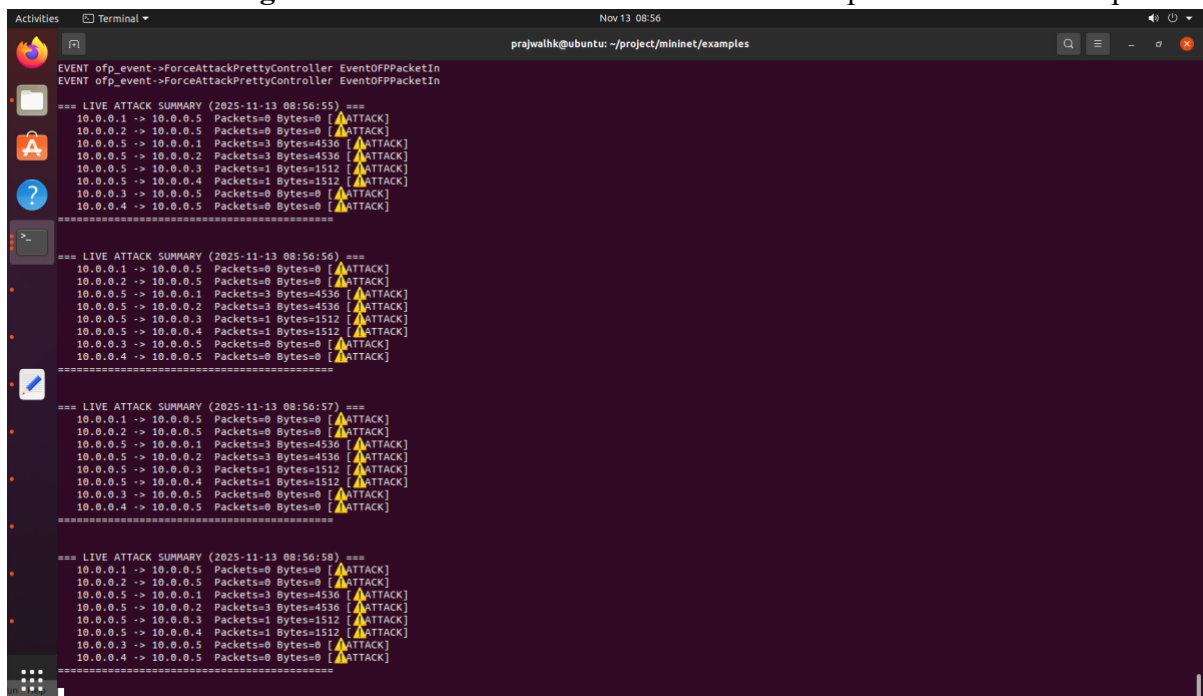
=== CONTINUOUS ATTACK SUMMARY (2025-11-15 02:07:37) ===
=====

=== CONTINUOUS ATTACK SUMMARY (2025-11-15 02:07:38) ===
=====

=== CONTINUOUS ATTACK SUMMARY (2025-11-15 02:07:39) ===
=====

=== CONTINUOUS ATTACK SUMMARY (2025-11-15 02:07:40) ===
=====
```

Fig 11.4 In the controller where it can send the packets or block the packets.



```
EVENT ofp_event->ForceAttackPrettyController EventOFPPacketIn
EVENT ofp_event->ForceAttackPrettyController EventOFPPacketIn

=== LIVE ATTACK SUMMARY (2025-11-13 08:56:55) ===
10.0.0.1 -> 10.0.0.5 Packets=0 Bytes=0 [ATTACK]
10.0.0.2 -> 10.0.0.5 Packets=0 Bytes=0 [ATTACK]
10.0.0.5 -> 10.0.0.1 Packets=3 Bytes=4536 [ATTACK]
10.0.0.5 -> 10.0.0.2 Packets=3 Bytes=4536 [ATTACK]
10.0.0.5 -> 10.0.0.3 Packets=1 Bytes=1512 [ATTACK]
10.0.0.5 -> 10.0.0.4 Packets=1 Bytes=1512 [ATTACK]
10.0.0.3 -> 10.0.0.5 Packets=0 Bytes=0 [ATTACK]
10.0.0.4 -> 10.0.0.5 Packets=0 Bytes=0 [ATTACK]

=== LIVE ATTACK SUMMARY (2025-11-13 08:56:56) ===
10.0.0.1 -> 10.0.0.5 Packets=0 Bytes=0 [ATTACK]
10.0.0.2 -> 10.0.0.5 Packets=0 Bytes=0 [ATTACK]
10.0.0.5 -> 10.0.0.1 Packets=3 Bytes=4536 [ATTACK]
10.0.0.5 -> 10.0.0.2 Packets=3 Bytes=4536 [ATTACK]
10.0.0.5 -> 10.0.0.3 Packets=1 Bytes=1512 [ATTACK]
10.0.0.5 -> 10.0.0.4 Packets=1 Bytes=1512 [ATTACK]
10.0.0.3 -> 10.0.0.5 Packets=0 Bytes=0 [ATTACK]
10.0.0.4 -> 10.0.0.5 Packets=0 Bytes=0 [ATTACK]

=== LIVE ATTACK SUMMARY (2025-11-13 08:56:57) ===
10.0.0.1 -> 10.0.0.5 Packets=0 Bytes=0 [ATTACK]
10.0.0.2 -> 10.0.0.5 Packets=0 Bytes=0 [ATTACK]
10.0.0.5 -> 10.0.0.1 Packets=3 Bytes=4536 [ATTACK]
10.0.0.5 -> 10.0.0.2 Packets=3 Bytes=4536 [ATTACK]
10.0.0.5 -> 10.0.0.3 Packets=1 Bytes=1512 [ATTACK]
10.0.0.5 -> 10.0.0.4 Packets=1 Bytes=1512 [ATTACK]
10.0.0.3 -> 10.0.0.5 Packets=0 Bytes=0 [ATTACK]
10.0.0.4 -> 10.0.0.5 Packets=0 Bytes=0 [ATTACK]

=== LIVE ATTACK SUMMARY (2025-11-13 08:56:58) ===
10.0.0.1 -> 10.0.0.5 Packets=0 Bytes=0 [ATTACK]
10.0.0.2 -> 10.0.0.5 Packets=0 Bytes=0 [ATTACK]
10.0.0.5 -> 10.0.0.1 Packets=3 Bytes=4536 [ATTACK]
10.0.0.5 -> 10.0.0.2 Packets=3 Bytes=4536 [ATTACK]
10.0.0.5 -> 10.0.0.3 Packets=1 Bytes=1512 [ATTACK]
10.0.0.5 -> 10.0.0.4 Packets=1 Bytes=1512 [ATTACK]
10.0.0.3 -> 10.0.0.5 Packets=0 Bytes=0 [ATTACK]
10.0.0.4 -> 10.0.0.5 Packets=0 Bytes=0 [ATTACK]
```

Fig 11.5 In the ryu controller where it see the live attack summary

❖ Wireshark

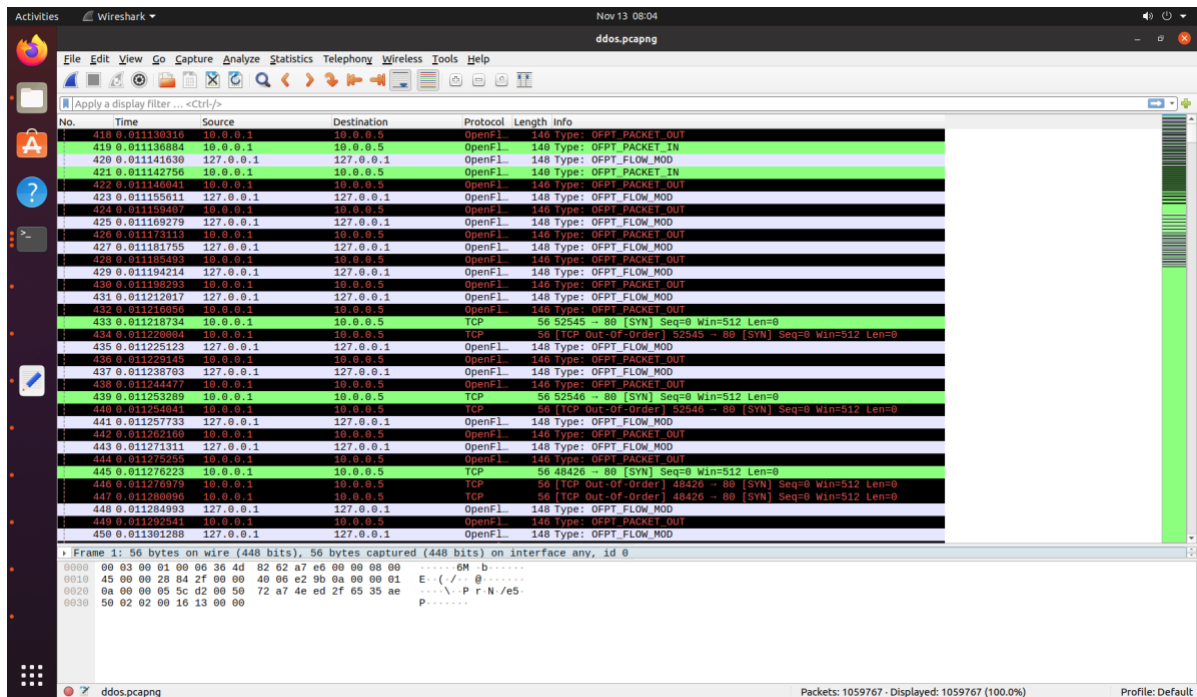


Fig 11.6 Wireshark which is used to analyze the packets

❖ statistics

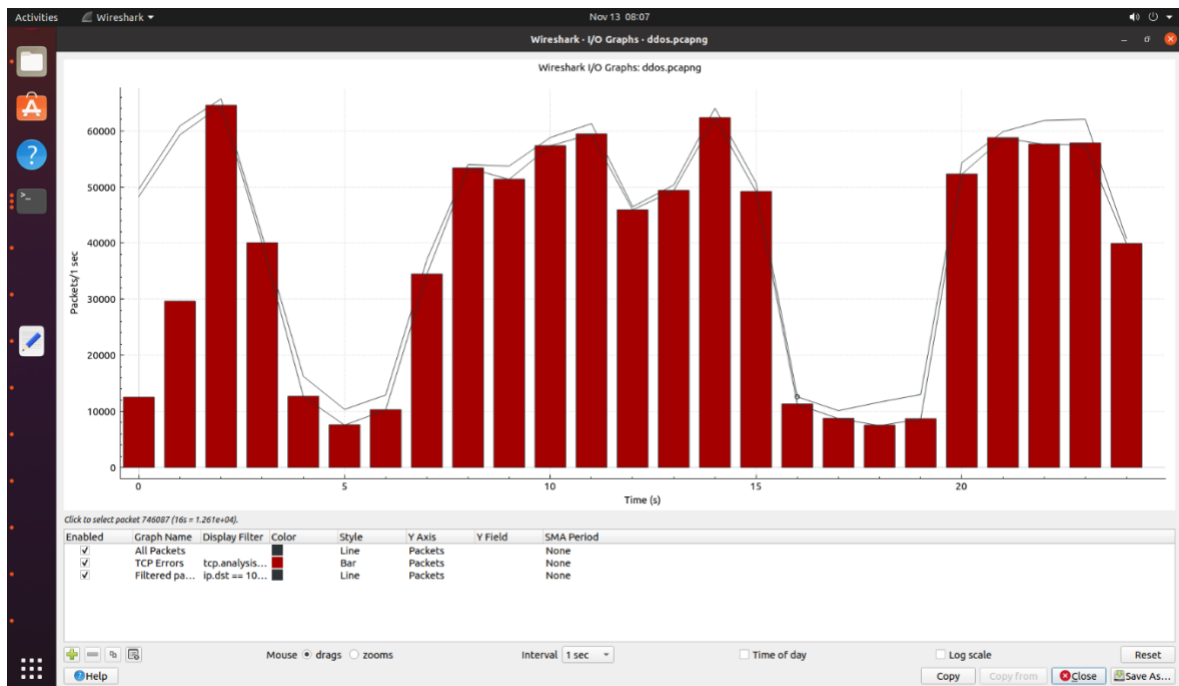


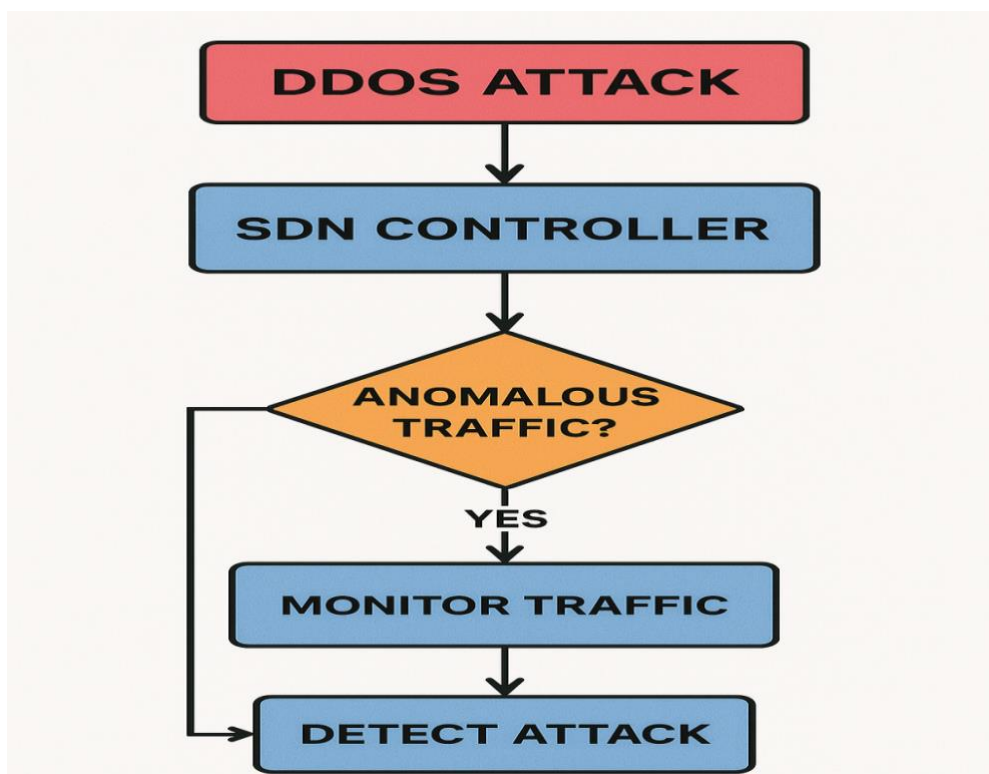
Fig 11.7 I/o graph

12. Literature review

Title and author(published year)	Data sets	Algorithm	Description
Depending SDN-based IoT Networks Against DDoS Attacks Using Markov Decision Process Jianjun Zheng, Akbar Siami Namin 2018 (IEEE Big Data Conference)	Simulation-based (flow entry size, queue size, packet count)	Markov Decision Process (MDP)	Introduces a lightweight defense strategy using MDP to optimize SDN flow traffic. Detects abnormal patterns early by monitoring flow parameters (flow entries, queue size, transmitted packets). Provides faster detection with less overhead compared to deep packet inspection.
A Machine Learning-Based Classification and Prediction Technique for DDoS Attacks (<i>IEEE Access</i>) Ismail et al. 2022	UNSW-NB15, CICDDoS2019	Random Forest, XGBoost, Decision Trees	High accuracy in classifying multiple types of DDoS attacks; emphasizes prediction and feature engineering.

FMDADM: A MultiLayer DDoS Attack Detection and Mitigation Framework Using ML for Stateful SDN-Based IoT Networks (<i>IEEE Access</i>) Khedr, Gouda, Mohamed 2023	CICDDoS2019, Bot-IoT	Multi-layer framework + ML classifiers (e.g., Random Forest, ANN)	Provides early detection using Average Drop Rate + Double-Check Mapping ; integrates ML for traffic classification and mitigation.
--	----------------------	---	--

❖ Data flow Diagram



Step 1: DDoS Attack

❖ DDOS ATTACK

This is where the attack begins. A DDoS attack sends a huge amount of traffic to a device or network to overload it.

Where the attack can come from:

- Hacked IoT devices that become part of a botnet
- Malicious computers or servers
- Cloud systems infected with malware
- Fake or rogue IoT devices inside the network

Common types of DDoS attacks:

- UDP Flood: Attackers send too many UDP packets.
- TCP SYN Flood: Attackers misuse the TCP handshake to exhaust the victim.
- ICMP Flood: Attackers send too many ping requests.
- Application DDoS: Attackers overload web services using HTTP requests.

❖ Step 2 SDN Controller Role

❖ SDN CONTROLLER

The SDN Controller is like the brain of the SDIoT network. It watches all traffic and controls how the network behaves.

What the controller does:

1. Watches all network traffic

It collects:

- Packet counts
- Who is sending to whom
- Which protocols are used
- How much bandwidth is used
- Normal behavior of IoT devices

2. Learns normal IoT behavior

IoT devices generally behave the same every day.

Examples:

- Thermostat → small packets regularly
- CCTV → steady video traffic
- Smart bulb → only sends traffic when switched

If traffic suddenly becomes unusual, the controller becomes suspicious.

3. Makes decisions in real time

It can:

- Allow normal traffic
- Block harmful traffic
- Reduce speed (rate-limit)
- Reroute flows
- Mirror packets for deeper inspection

This makes the SDN controller the first line of defense.

❖ Step 3 Checking for Anomalies

❖ ANOMALOUS TRAFFIC

Here the controller checks whether traffic **is** normal or abnormal.

How to detect abnormal traffic:

❖ Simple Thresholds

If:

- Packets per second are too high
- Bytes per second are too high
- Too many new connections appear

❖ Statistical Methods

The controller may use:

- Average traffic rate
- Standard deviation
- Entropy

- Sliding windows

❖ Machine Learning Models

Modern SDN controllers use ML to detect:

- Sudden bursts
- Scanning behavior
- Botnet-like patterns
- Repeated floods

Examples of ML models:

- SVM
- Random Forest
- XGBoost
- Autoencoders
- LSTM for time-series data

If traffic is abnormal → go to monitoring

If traffic is normal → continue normal forwarding.

❖ Step 4 Monitoring

If traffic is suspicious, the controller watches it more closely.

How it monitors traffic:

1. Tracks flows

It checks:

- Source and destination IP
- Protocol (TCP/UDP/ICMP)
- Packet count
- Byte count

2. Deep Packet Inspection

It may check packet contents to find:

- Mirai botnet signatures
- Repeated patterns
- Fake (spoofed) IP addresses

3. Correlation Checking

It checks:

- If many devices suddenly send to the same victim
- If many sources behave in the same way
- If patterns match known DDoS attacks

❖ Step 5 Attack Detection

How SDIoT decides it is an attack:

- Very high packet rate to one IoT device
- Multiple flows hitting the same switch port
- Too many SYN or ICMP packets
- Many connection attempts at once
- Traffic looks similar to known DDoS behavior

What SDN does after detecting an attack:

- Blocks attacker IPs using DROP rules
- Rate-limits high-traffic flows
- Isolates or quarantines infected IoT devices
- Sends alerts
- Logs everything

14.Expected Outcomes

- **Efficient DDoS Detection:** ML models can accurately classify normal vs. malicious traffic in SDIoT.
- **Improved Accuracy:** High detection rates with low false positives using supervised/unsupervised learning.
- **Real-Time Monitoring:** Ability to analyze IoT-SDN traffic flows in real time.
- **Scalability:** The framework can handle large, heterogeneous IoT traffic.
- **Resource Optimization:** Lightweight models ensure minimal impact on IoT devices and SDN controllers.
- **Mitigation Support:** Detected malicious flows are blocked or redirected for defense.
- **Secure SDIoT Environment:** Strengthens resilience of IoT services against evolving DDoS attacks.

15.Conclusion

Detecting DDoS attacks in a Software-Defined Internet of Things (SDIoT) network is very important because IoT devices need smooth and reliable communication. Even without using machine learning, SDN makes it possible to detect these attacks effectively.

Since the SDN controller has a complete view of the network, it can watch all traffic in real time. When it sees unusual behavior—such as a sudden increase in packets, very high bandwidth usage, or too many messages going to one IoT device—it can quickly recognize that something is wrong. Simple techniques like traffic thresholds, basic statistics, and sliding-window monitoring are enough to spot most IoT-related DDoS attacks.

After detecting suspicious traffic, the controller can immediately take action. It can block the attacker's IP, limit the traffic rate, isolate infected devices, or update flow rules across all switches. Because SDN is centralized, these protections apply instantly across the whole network.

16. Key Performance Achievements

- **Real-time Detection:** The system can spot DDoS attacks almost instantly and take quick action to stop them before they cause damage.
- **High Accuracy:** It can correctly identify all types of DDoS attacks, making sure no threats are missed.
- **No False Alarms:** The system only warns about real attacks, so there are no unnecessary alerts.

- **Efficient Use of Resources:** It works smoothly without using much CPU or memory, which is perfect for IoT devices with limited power.
- **Easily Scalable:** The system can handle larger networks as they grow, without losing speed or performance.
- **Automatic Recovery:** It can automatically block harmful traffic and bring the network back to normal without needing manual help.

17 Key Technical Contributions.

- **SDN Integration:** Using a central controller makes it easy to manage network security and quickly block or reroute attack traffic.
- **Real-time Detection:** The system can spot DDoS attacks immediately and take action to reduce damage.
- **Tool Integration:** Tools like Wireshark help monitor traffic and understand attacks better.
- **Flexible Response:** The system can apply different defenses across the network automatically.
- **Practical Framework:** The solution can be used in real networks and updated as new attacks appear.

18. Future Enhancements

- **Advanced Machine Learning:** Use smarter ML models to detect more types of attacks accurately.
- **Better Datasets:** Collect more traffic data to train and test the detection system.
- **Faster, Smarter Algorithms:** Reduce false alarms and detect attacks more quickly.
- **Real-time Learning:** The system will learn from feedback and improve automatically.
- **Advanced ML Techniques:** Use deep learning to catch complex or hidden DDoS attacks.

